

ZONT API

MQTT + WebSocket - техническая документация v2

Основа документа: официальный PDF «Формат обмена сообщениями с контроллерами ZONT через JSON», MQTT-документация ZONT, а также наблюдаемые ответы конкретного контроллера. Где сведения получены из практических наблюдений, это отмечено отдельно.

1. Архитектура и область применения

WebSocket API предназначен для прямого JSON-обмена с контроллером. Состояния объектов и команды управления используют ту же модель данных, что и MQTT-протокол ZONT. Это позволяет строить локальные интеграции, в том числе Home Assistant, без промежуточного облачного API.

Home Assistant / локальная интеграция | | WebSocket JSON v ZONT controller | +---- внутренние объекты: датчики / реле / насос / контуры | +---- MQTT client -> внешний MQTT broker (если функция доступна в модели/прошивке) | +---- облачный канал ZONT

Важно: наличие MQTT-клиента и возможность его настройки зависят от модели и прошивки. WebSocket API документирован отдельно и не означает автоматическое наличие пользовательской настройки MQTT.

2. WebSocket API

2.1. Адрес подключения

ws://<ip-address>/ws wss://<ip-address>/ws # если контроллер работает по HTTPS

В официальном документе адрес записан как HTTP/HTTPS URL, однако транспортом является WebSocket; в клиентских библиотеках используются схемы ws:// и wss://.

2.2. Авторизация

```
{ "user": "user_name", "pass": "user_password" }
```

```
{ "auth": 200 } # успешно { "auth": 401 } # ошибка авторизации
```

Используются имя и пароль пользователя из настроек пользователей контроллера.

2.3. Получение списка ID объектов

```
{ "req_ids": 16 } -> { "ids": [20494, 20496, 8362] }
```

Поле type соответствует типу объекта в конфигурации. Для запроса объектов любого поддерживаемого типа официальный документ указывает type=255.

2.4. Запрос состояния

```
{ "id": 20496, "req_state": 0 }
```

Ответ содержит id, type, name и набор полей, зависящий от типа объекта. Официальная спецификация прямо указывает, что поля состояния те же, что и в MQTT-протоколе ZONT.

2.5. Управление объектом

```
{ "id": 20496, "cmd": 3130 } -> { "id": 20496, "cmdres": <result> }
```

Command имеет тот же формат, что команда cmd в MQTT. В WebSocket дополнительно указывается id объекта.

2.6. Управление устройством / терминальные команды

```
{ "scmd": "#S7?" } -> { "scmdres": "#S7:H2000PR0..." }
```

Это единственный конкретный пример системной команды, явно приведенный в официальном JSON API документе. Другие #S-команды ниже отмечены как наблюдаемые на реальном устройстве, а не как полный официальный справочник.

3. Типы объектов

type	Официальное описание
0	Аналоговый вход
1	Датчик температуры цифровой DS18...
2	Охранная зона
6	Адаптер ЦШ
8	Радиодатчик
10	Элемент управления пользовательский
14	Реле
15	Смеситель
16	Контур отопления
17	Насос
27	NTC термодатчик
255	Любой тип (для req_ids)

В конфигурациях конкретных устройств могут встречаться дополнительные служебные типы. Например, в исследованной конфигурации были замечены type 20 и 21. Они не включены в таблицу выше, поскольку отсутствуют в предоставленной официальной таблице WebSocket API.

4. Подтипы stype

4.1. WEB_ELEMENT - type 10

stype	Назначение
0	Статус
1	Простая кнопка
2	Сложная кнопка
3	Аналоговый регулятор

4.2. HEATING_CIRCUIT - type 16

stype	Назначение
0	Котловой контур
1	ГВС
2	Охладительный контур
3	Контур потребителя

4.3. ANALOG_INPUT - type 0

stype	Назначение
0	Аналоговый вход (без пресета)
1	Датчик давления 5 бар
2	Датчик давления 12 бар
3	Датчик открытия двери
4	ИК-датчик движения с контролем шлейфа

stype	Назначение
5	Датчик дыма
6	Датчик протечки
7	ИК-датчик движения без контроля шлейфа
8	Комнатный термостат
9	Авария котла +
10	Авария котла -
11	Вход зажигания
12	Датчик скорости
13	Датчик оборотов двигателя
14	Дискретный вход
15	Тревожная кнопка
16	Датчик расхода топлива
17	Датчик влажности
18	Датчик давления 6 бар
19	Дискретный вход НР
20	Дискретный вход НЗ
21	Датчик давления 10 бар (с версии 475)

4.4. RADIO_SENSOR - type 8

stype	Назначение
2	Брелок 3-кнопочный
3	Брелок 4-кнопочный
4	Радиореле блокировки
5	Радиотермометр
6	Радиодут (термин из исходного документа)
7	Модуль капота: концевик, термометр, замок, сирена
8	Радиометка
10	Радиодатчик протечки
11	Радиодатчик движения
12	Радиодатчик удара
13	3 температурных датчика + концевой контакт
14	Радиодатчик расхода электроэнергии
15	Внешний радиодатчик температуры
16	Радиодатчик расхода воды и газа
17	Радиорозетка 220 V
18	Радиодатчик температуры и влажности
19	Радиометка с акселерометром

stype	Назначение
20	Радиометка с аккумулятором
23	Радиопанель / радиотермостат

5. Поля состояния объектов

Набор полей зависит от type и stype. Ниже приведены поля, подтвержденные предоставленными протоколами и наблюдаемыми ответами контроллера.

Поле	Смысл	Применение
id	Идентификатор объекта	Все объектные ответы
type	Тип объекта	Все объектные ответы
stype	Подтип	Часть типов: 0, 8, 10, 16
name	Имя объекта	Большинство объектов
t	Температура	Температурные датчики
v	Значение аналогового входа	type 0
a	Активность/доступность	Датчики и входы
trig	Состояние триггера	Датчики/входы
c	Текущее значение/температура контура	type 16; фактический смысл зависит от stype
s	Уставка контура	type 16
m	Текущий режим - наблюдаемое поле	type 16
m_id	ID режима - наблюдаемое поле	type 16
f	Флаг/состояние - зависит от объекта	Несколько типов
water	Температура теплоносителя	type 6, адаптер котла
dhw	Температура ГВС	type 6
state	Состояние адаптера	type 6
err	Код ошибки	type 6

5.1. Примеры реальных ответов

```
{ "id": 8196, "type": 1, "name": "Погода из интернета", "t": 9.9, "a": 1, "trig": 0 }
```

```
{ "id": 20496, "type": 16, "stype": 3, "name": "Отопление", "m": "heat", "m_id": 20501, "c": 36.0, "s": 41.0, "f": 0 }
```

Примечание: в наблюдаемом WebSocket-ответе контура значения c и s уже представлены в °C. Нельзя автоматически считать все поля c/s децикельвинами только по их имени. Единицы необходимо трактовать по конкретному типу сообщения и MQTT-описанию.

```
{ "id": 4097, "type": 6, "name": "Navien", "water": 36.0, "dhw": 35.0, "state": 0, "err": 0 }
```

6. MQTT и соответствие WebSocket

Официальная WebSocket-спецификация отсылает к MQTT-протоколу для содержимого state и Command. Практически это означает, что объектная семантика общая, а транспорт различается.

Операция	MQTT	WebSocket
Чтение состояния	Публикации по топикам протокола	{ "id": id, "req_state": 0 }
Управление объектом	cmd в MQTT	{ "id": id, "cmd": Command }
Результат команды	Ответ/публикация MQTT	{ "id": id, "cmdres": res }

Операция	MQTT	WebSocket
Системная команда	Не является базовой объектной командой	{"scmd": "#S..."}

6.1. Температурные команды и DK

В MQTT-документации часть команд температуры задается в децикельвинах (0,1 K). Для таких команд используется преобразование:

$$DK = \text{round}((^{\circ}C + 273.0) * 10) \quad ^{\circ}C = DK / 10 - 273.0$$

Пример: 40.0 °C -> 3130 DK

Не следует применять DK к каждому температурному полю ответа: в WebSocket-state конкретного контроллера часть температур приходит уже в °C.

7. Терминал и scmd

Официально подтвержден сам механизм scmd и пример #S7?. Ниже перечислены команды, которые были фактически зафиксированы в предоставленной WebSocket-сессии. Это не полный терминальный справочник.

Команда	Статус источника	Наблюдаемый смысл
#S7?	Официальный пример API	Идентификация/информация об устройстве
#S4?	Зафиксировано в сессии	Ответ содержал параметры GSM/оператора
#S198?	Зафиксировано в сессии	Сетевые/Wi-Fi данные
#S205?	Зафиксировано в сессии	Сетевой параметр; ответ #S205:!
#S208?	Зафиксировано в сессии	Wi-Fi credentials/status; содержит чувствительные данные

Команды вроде #R, #TIME?, #LOG?, #S250 и т.п. намеренно не включены: предоставленные официальные документы и логи их не подтверждают.

8. Рекомендуемая схема интеграции Home Assistant

```
HA custom component | +-- connect ws(s)://zont/ws +-- auth {user, pass} +-- req_ids(type) +-- req_state(id) +-- maintain object registry by id +-- push state changes into HA entities +-- send {id, cmd} for writes +-- optional scmd service for diagnostics
```

Для первой версии интеграции целесообразно поддерживать type 1 и 27 как sensor, type 14 как switch, type 16 как climate/sensor в зависимости от stype, type 6 как набор диагностических sensor, а type 17 как switch или fan-like entity после проверки фактических state-полей.

9. Практические замечания

Тема	Рекомендация
Reconnect	При обрыве WebSocket делать экспоненциальное переподключение и повторную авторизацию.
Bootstrap	После auth запросить req_ids для поддерживаемых типов, затем req_state каждого ID.
Push	Не использовать polling, если устройство само присылает обновления; req_state нужен для bootstrap/перепроверки.
Unknown fields	Сохранять неизвестные поля как diagnostics/attributes, не отбрасывать их на этапе парсинга.
cmdres	Коррелировать результат команды по id; не считать запись успешной только по факту отправки кадра.
Secrets	Не логировать pass, Wi-Fi credentials из #S208 и иные чувствительные ответы scmd.

10. Источники и уровень достоверности

Официально подтверждено: адрес /ws, авторизация, req_ids, req_state, type 0/1/2/6/8/10/14/15/16/17/27/255, stype для 0/8/10/16, формат {id,cmd}, cmdres, scmd и пример #S7?.

Подтверждено практическими данными конкретного контроллера: поля ответов для temperature sensor, analog input, boiler adapter и heating circuit; команды #S4?, #S198?, #S205?, #S208?.

Не включено как достоверное: неподтвержденные списки терминальных команд, предполагаемые MQTT #S-параметры и модели внутренних сервисов.